NP-Completeness

Design and Analysis of Algorithms

Sujoy Sarkar

1. A Pattern in Problems We Studied

Polynomial ("Good News")

- Merge / Quick Sort ($n \log n$)
- Closest Pair / Convex Hull ($n \log n$)
- Prim's / Kruskal ($E \log V$)
- Fractional Knapsack ($n \log n$)
- Huffman Coding ($n \log n$)
- Optimal Merge ($n \log n$)
- Matrix Chain (n^3)
- OBST (n^3)

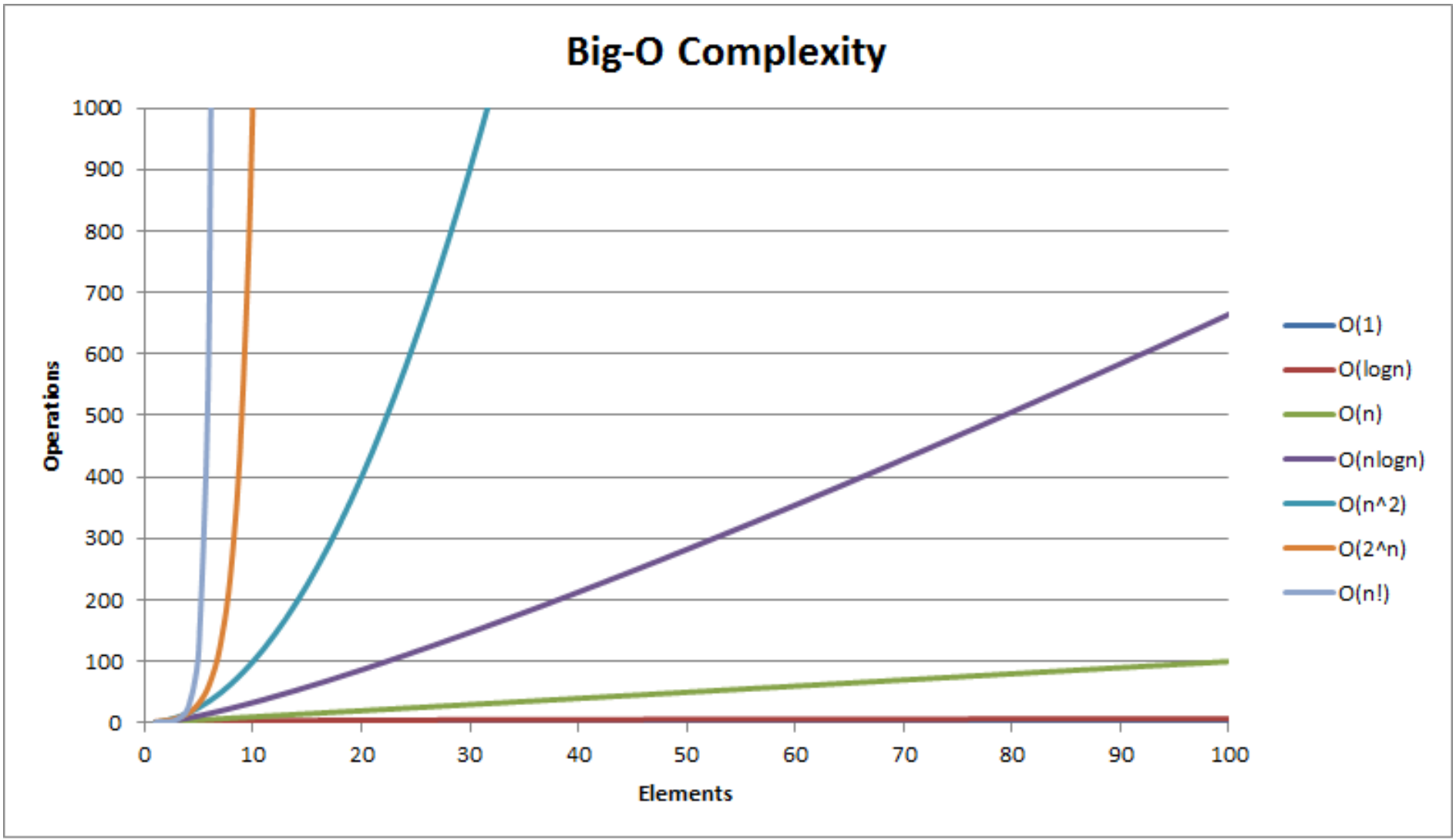
Hard ("Bad News")

- 0/1 Knapsack (2^n)
- Sum of Subsets (2^n)
- N-Queens ($n!$)
- Permutations ($n!$)
- Hamiltonian Cycle ($n!$)
- TSP ($n^2 2^n$)

Observation

Polynomial vs Exponential growth $\rightarrow$ huge difference in scalability

Polynomial vs Exponential



2. A Subtle but Important Difference

Shortest Path

- Efficient algorithms exist
- (Dijkstra, Bellman-Ford)

Longest Path

- No known efficient solution
- Related to Hamiltonian Path

Key Insight

A small change in problem definition can make it **very hard**

Why should we care?

Knowing that they are hard lets you stop beating your head against a wall trying to solve them

- **Use a heuristic** come up with a method for solving a reasonable fraction of the common cases
- **Solve approximately** come up with a solution that you can prove that is close to right
- **Use an exponential time solution** if you really have to solve the problem exactly and stop worrying about finding a better solution

3. A New Perspective: Verification

Sudoku: Solve vs Verify

SOLVE

Hard for Computer

(Requires Search / Backtracking)

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4				8				1
7				2				
7						2		6
	6					2	8	
	4	1	8	9		5		

Exponential Time

No known Polynomial Algorithm

Key Idea:
Hard to solve ≠ Hard to verify

VERIFY

Easy for Computer

(Just Check Rows, Columns, Boxes)

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

Verification Steps ($O(n^2)$):

- ✓ Check all rows → 1 to 9
- ✓ Check all columns → 1 to 9
- ✓ Check all 3×3 boxes → 1 to 9

Polynomial Time!

Why Verification? Decision Problems

Decision vs Optimization

- **Decision Problems:** Answer is YES or NO
- **Optimization Problems:** Find the best solution

Key Idea

- Many optimization problems can be converted into decision problems

Example: Traveling Salesman Problem

- Optimization: Find the shortest tour
- Decision: Is there a tour of cost $\leq k$?

Why this matters

Verification is meaningful for **decision problems**

3. A New Perspective: Verification

Key Question

Can we verify a given solution quickly?

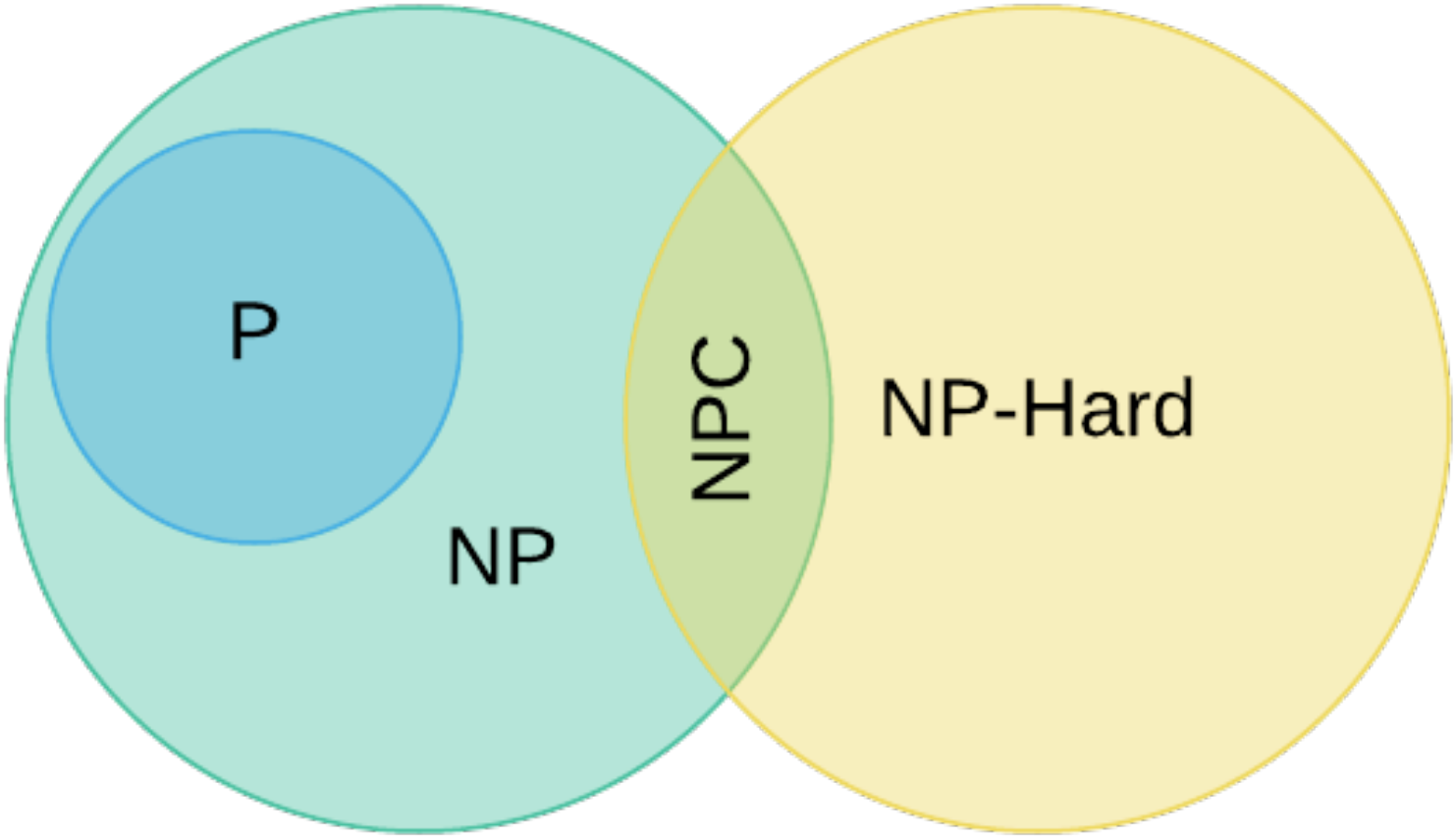
Examples

- Hamiltonian Cycle $\rightarrow$ check if valid cycle
- TSP $\rightarrow$ compute total cost of a given tour
- N-Queens $\rightarrow$ check conflicts in placement

Key Idea

NP = Problems where solutions can be verified efficiently

4. P and NP Class



4. The Big Question: P vs NP

Class P

- Problems we can **solve efficiently**
- Examples:
  - Sorting, Binary Search
  - MST (Prim's, Kruskal)

Class NP

- Solutions can be **verified efficiently**
- Examples:
  - TSP (given tour)
  - Hamiltonian Cycle

The Central Question

If we can verify quickly, can we also solve quickly?

$$\textbf{Is P} = \textbf{NP ?}$$

5. NP-Complete Problems

Key Idea

Some problems in NP appear to be the **hardest**

Informal Definition

A problem is **NP-Complete** if:

- It is in NP (verification is efficient)
- It is as hard as every problem in NP

Key Insight

If one NP-Complete problem is solved efficiently,
then **all NP problems can be solved efficiently**

Examples of NP-Complete Problems

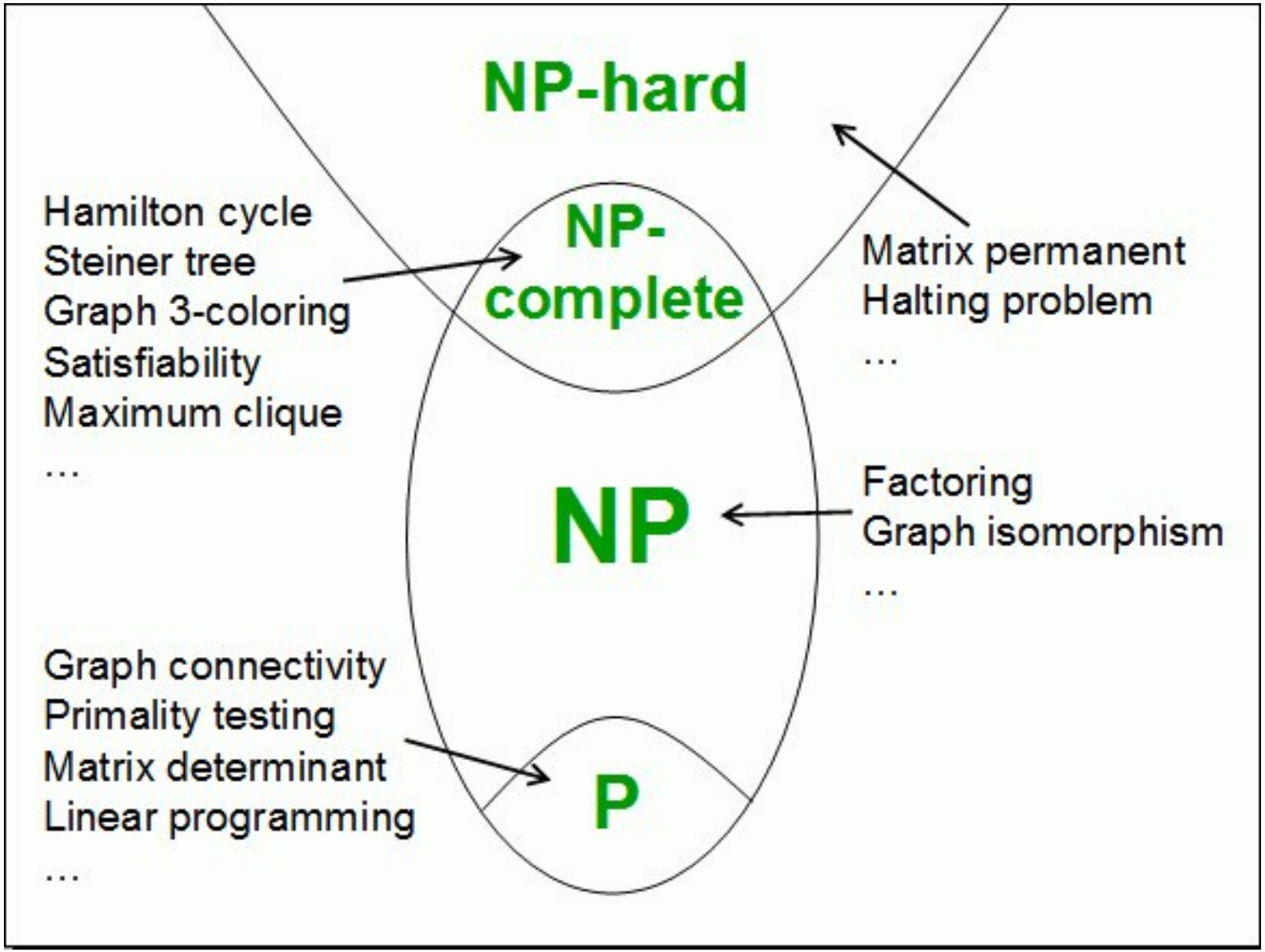
From Our Syllabus

- Traveling Salesperson Problem (TSP)
- Hamiltonian Cycle
- 0/1 Knapsack (decision version)

What does this mean?

- These problems are **equally hard**
- If we solve one efficiently $\rightarrow$ all become easy

Complexity Classes



6. Reduction: The Core Idea

What is Reduction?

Transforming one problem into another

Intuition

- Suppose we have two problems: A and B
- Convert an instance of A into an instance of B
- Solve B and use it to solve A

Key Idea

If $A \rightarrow B$ and B is easy, then A is also easy

Reduction in NP-Completeness

How do we prove a problem is hard?

- Start with a known NP-Complete problem (A)
- Reduce it to a new problem (B)

$$A \text{ (Hard)} \longrightarrow B \text{ (New Problem)}$$

Conclusion

- If A reduces to B, then B is also hard

Important

- Direction matters: $A \rightarrow B$
- Not $B \rightarrow A$

Example: Subset Sum Problem

Subset Sum

Given numbers $\{a_1, a_2, \ldots, a_n\}$ and a target T:

Is there a subset whose sum is exactly T?

Example

Numbers: $\{3, 5, 7, 10\}$, Target: $T = 15$

Possible subset: $\{5, 10\} \Rightarrow$ sum $= 15$

Observation

We try many subsets $\Rightarrow$ exponential possibilities

Knapsack Problem

0/1 Knapsack

- Each item has weight w_i and value v_i
- Capacity $= W$

Maximize total value without exceeding weight

Key Idea

Choose items such that:

$$\sum w_i \leq W$$

and value is maximized

Reduction: Subset Sum $\rightarrow$ Knapsack

Reduction Idea

- For each number a_i:
  - Set weight $w_i = a_i$
  - Set value $v_i = a_i$
- Set capacity $W = T$

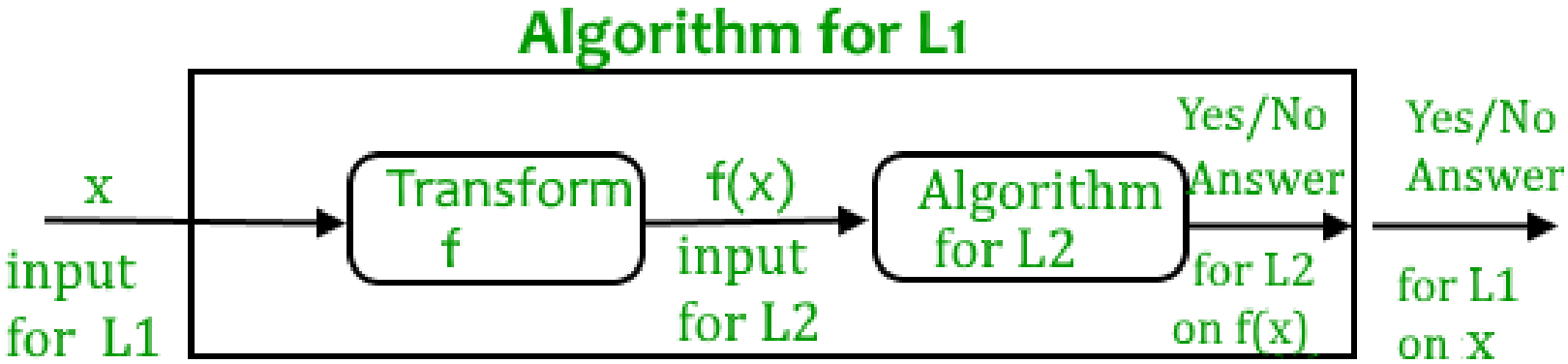
What happens?

- Finding subset with sum T
- $\equiv$ Filling knapsack exactly to capacity W

Conclusion

- If we can solve Knapsack efficiently
- Then we can solve Subset Sum efficiently

Reduction


Algorithm for L1
x
input
for L1
Transform
f
f(x)
input
for L2
Algorithm
for L2
Yes/No
Answer
for L2
on f(x)
Yes/No
Answer
for L1
on x

7. The First NP-Complete Problem: SAT

What is SAT?

Given a Boolean formula:

$$(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3)$$

Is there an assignment that makes it TRUE?

Example

If $x_1 = \mathrm{TRUE}$, the formula becomes TRUE

Key Fact

SAT was the **first NP-Complete problem**

Cook-Levin Theorem

The Theorem (1971)

The Boolean Satisfiability Problem (SAT) is NP-Complete.

- Every language in NP is **polynomial-time reducible** to SAT ($L \leq_p$ SAT).
- Proven independently by Stephen Cook and Leonid Levin.

Significance

- If a polynomial-time algorithm exists for SAT, then $P = NP$.
- Provides the "anchor" for proving other problems are NP-Complete via reduction.

8. Chain of NP-Complete Problems

Idea

We reuse known NP-Complete problems

$$\text{SAT} \rightarrow \text{3-SAT} \rightarrow \text{Clique} \rightarrow \text{Vertex Cover} \rightarrow \text{Subset Sum}$$

Key Understanding

- Hardness flows through reductions

Why This Chain Matters

Important

- Once NP-Complete, always reusable

Final Insight

NP-Complete problems form a network of hardness

Formal Definition of NP-Completeness

A problem B is NP-Complete if:

- $B \in \mathrm{NP}$
  (solutions can be verified efficiently)
- A known NP-Complete problem A reduces to B
  (i.e., $A \rightarrow B$)

Interpretation

- B is at least as hard as A
- Since A is already hard, B is also hard

Strategy

- Start from known NP-Complete problem
- Reduce it to the new problem

Worked Example: 3-SAT

3-SAT Problem

A Boolean formula in CNF where each clause has exactly 3 literals

Example:

$$(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_4)$$

Goal

Is there an assignment of TRUE/FALSE that satisfies the formula?

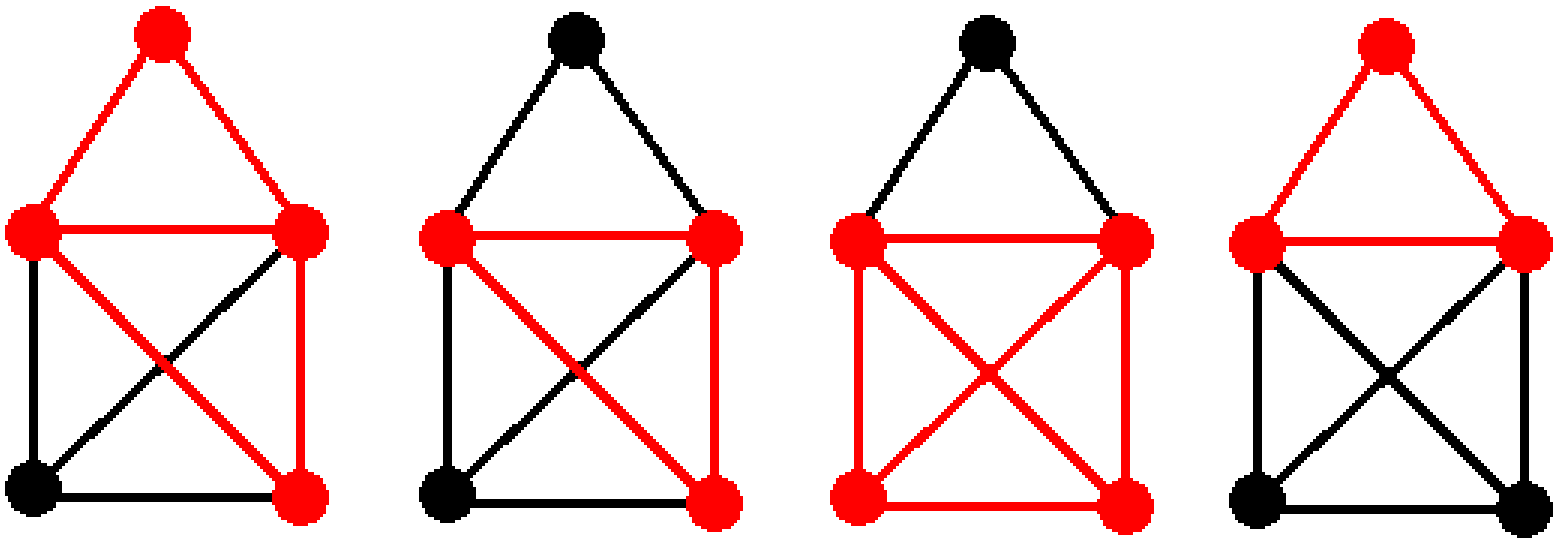
Clique Problem

Clique (Decision Problem)

Given a graph G and an integer k:

Does G contain a clique of size k?

Clique: A set of vertices where every pair is connected



not a clique | non-maximal clique | maximal clique | maximal clique

Reduction: 3-SAT $\rightarrow$ Clique

Goal: Convert 3-SAT into a graph problem

- Each clause $\rightarrow$ a group of nodes
- Each literal $\rightarrow$ one node

Connection Rule

Connect two nodes only if their literals are not conflicting

Mapping Between 3-SAT and Clique

Key Idea

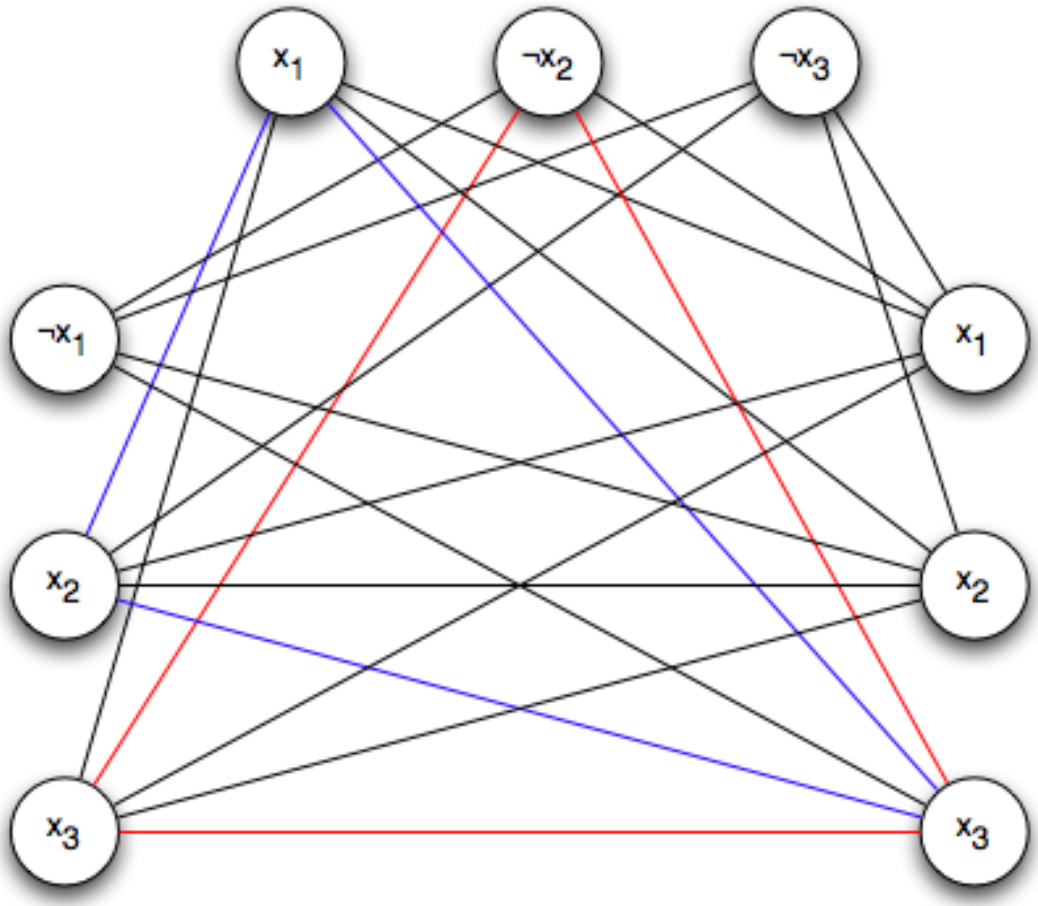
- Choose one literal from each clause
- Ensure chosen literals are consistent

Interpretation

A clique represents a consistent selection of literals

Mapping Between 3-SAT and Clique

$\Phi_1 = (x_1 \vee \neg x_2 \vee \neg x_3)$



$\Phi_2 = (\neg x_1 \vee x_2 \vee x_3)$

$\Phi_3 = (x_1 \vee x_2 \vee x_3)$

Conclusion of Reduction

Result

3-SAT reduces to Clique

- 3-SAT is NP-Complete
- Therefore, Clique is also NP-Complete

Key Insight

Hardness is transferred through reduction

Conclusion of Reduction

Result

3-SAT reduces to Clique

- 3-SAT is NP-Complete
- Therefore, Clique is also NP-Complete

Key Insight

Hardness is transferred through reduction

Other Important NP-Complete Problems

Graph Problems

- Clique (Decision)
- Vertex Cover (Node Covering)
- Hamiltonian Cycle

Scheduling Problems

- Exam Scheduling
- Task Scheduling with constraints

Code Generation Problems

- Register Allocation (Graph Coloring)

Where Do These Problems Appear?

- Graph theory (Clique, Vertex Cover)
- Optimization problems (TSP, Knapsack)
- Scheduling and resource allocation
- Compiler design (Register Allocation)

Insight

NP-Complete problems arise in many real-world applications

Vertex Cover Problem

Vertex Cover (Decision)

Given a graph G and integer k:

Is there a set of k vertices that covers all edges?

Meaning: Every edge has at least one endpoint in the set

Reduction: Clique $\rightarrow$ Vertex Cover

Idea: Use complement graph

- Start with graph G
- Construct complement graph $\overline{G}$

Key Relation

Clique of size k in G

$$\Longleftrightarrow$$

Vertex Cover of size $n - k$ in $\overline{G}$

Conclusion

Result

Clique reduces to Vertex Cover

- Clique is NP-Complete
- Therefore, Vertex Cover is also NP-Complete

Key Idea

We reuse known NP-Complete problems to prove others hard

Conclusion

What We Learned

- Some problems are efficiently solvable (P)
- Some problems are hard but solutions can be verified (NP)
- NP-Complete problems are the hardest in NP

Key Tools

- Reduction is used to prove hardness
- Cook's Theorem: All NP problems reduce to SAT

Final Insight

If any NP-Complete problem is solved efficiently,
then $\mathrm{P} = \mathrm{NP}$